AI-Powered Business Insights Assistant

Project Documentation

This document covers the complete tech stack, system architecture, data flow, and critical setup steps for the AI-Powered Business Insights Assistant — a full-stack GenAI application built with Streamlit, LangChain, Groq, and ChromaDB.

1. Tech Stack

Layer	Technology	Version	Purpose
UI	Streamlit	1.35.0	Web interface, file upload, tabs
LLM	Groq — Llama 3.3 70B	API	Free AI chat and reasoning
Orchestration	LangChain	0.3.0	Chain, memory, prompts
RAG Chain	ConversationalRetrievalChain	0.3.0	Retrieval-augmented chat
Vector DB	ChromaDB	0.5.0	Stores and searches embeddings
Embeddings	HuggingFace all-MiniLM-L6-v2	Local	Converts text to vectors
Data Processing	Pandas	2.3.3	Tabular data analysis
Visualizations	Plotly	6.7.0	Interactive charts
PDF Reading	PyPDF	4.2.0	Extract text from PDFs
Excel Reading	OpenPyXL	3.1.4	Read .xlsx files
Text Splitting	LangChain Text Splitters	0.3.0	Chunk documents for RAG
NLP	NLTK	3.8.1	Text preprocessing
Config	python-dotenv	1.0.1	Load .env variables
Language	Python	3.11.9	Runtime

2. System Architecture

The application is structured in four layers, each with a clear responsibility:

Layer	Components	Responsibility
Presentation	Streamlit UI Sidebar, Chat tab, Analytics tab	User interaction, file upload, display
Orchestration	LangChain ConversationalRetrievalChain + Memory	Connects LLM, retriever, and memory
Intelligence	Groq LLM (Llama 3.3 70B) HuggingFace Embeddings	Generates answers, creates vectors
Storage	ChromaDB (disk) Pandas DataFrames (memory)	Persists vectors, holds tabular data

3. What Really Happens — Data Flow

Step 1: File Upload

When you upload a file through the Streamlit sidebar:

- The file is saved to a temporary path on disk.
- document_processor.py detects the file type (CSV, Excel, PDF, or text).
- For CSV/Excel: each row becomes a LangChain Document, plus a summary document with column stats.
- For PDF: each page becomes a separate Document.
- For text: the full content becomes one Document.

Step 2: Text Splitting

All documents are passed through RecursiveCharacterTextSplitter:

- Chunk size: 1000 characters with 150 character overlap.
- Overlap ensures context is not lost at chunk boundaries.
- A 480-row CSV produces roughly 500+ chunks after splitting.

Step 3: Embedding

Each text chunk is converted to a 384-dimensional vector:

- HuggingFace all-MiniLM-L6-v2 runs entirely on CPU — no API call needed.
- Similar text produces vectors that are close together in vector space.
- Vectors are stored in ChromaDB and persisted to the ./vectorstore/ folder on disk.

Step 4: Chat Query

When you type a question:

- LangChain checks conversation history and rephrases the question as standalone if needed.
- The question is embedded using the same HuggingFace model.
- ChromaDB finds the 5 most similar chunks using cosine similarity.
- The chunks + question are sent to Groq (Llama 3.3 70B) as a prompt.
- The LLM returns a grounded answer based only on the retrieved context.
- The answer and source references are displayed in the chat.

Step 5: Analytics Tab

For tabular files (CSV/Excel), the analytics tab works independently of the RAG pipeline:

- Pandas computes KPIs: sum, mean, min, max for each numeric column.
- Plotly auto-generates up to 4 charts: bar, line, histogram, and scatter.

- A correlation heatmap shows relationships between numeric columns.
- The custom chart builder lets you pick any two columns and chart type.

4. Project File Structure

File	What it does
app.py	Entry point. Sets up Streamlit page config and renders sidebar + tabs.
components/sidebar.py	File uploader. Processes files, builds vector store, initializes chain.
components/chat.py	Chat UI. Renders message history, handles input, shows source references.
components/analytics_view.py	Analytics UI. KPI cards, auto charts, custom chart builder, data preview.
utils/document_processor.py	Loads CSV, Excel, PDF, and text files into LangChain Documents.
utils/vector_store.py	Manages ChromaDB: create, load, and add documents.
utils/rag_chain.py	Builds the ConversationalRetrievalChain with Groq LLM and memory.
utils/analytics.py	Pandas KPI computation and Plotly chart generation.
sample_data/generate_sample.py	Generates a 480-row sales CSV for testing.
.env	Stores API keys and config. Never commit this to Git.
requirements.txt	All Python package dependencies with versions.

5. Critical Setup Steps

Python Version

This project requires Python 3.11.9. Python 3.13 and 3.14 are too new — key packages like numpy and streamlit do not have pre-built wheels for them and will fail to install.

- Download Python 3.11.9 from python.org/downloads/release/python-3119/
- Always create the virtual environment using the full path to python 3.11.

Virtual Environment

Always activate the virtual environment before running the app. Without it, packages installed inside venv are not available.

```
& "C:\...\Python311\python.exe" -m venv venv
venv\Scripts\activate
```

Package Installation

Use --only-binary :all: to avoid source compilation errors on Windows. Key version fixes applied in this project:

Package	Issue	Fix Applied
langchain-chroma	0.1.1 not available for Python 3.11	Changed to 0.1.2, later 0.1.4
pandas	2.2.2 tried to compile from source	Changed to >= 2.2.3
tiktoken	0.7.0 not available for Python 3.11	Changed to >= 0.12.0
groq	proxies keyword error with httpx	Pinned groq==0.9.0 httpx==0.27.0
llama3-8b-8192	Model decommissioned by Groq	Switched to llama-3.3-70b-versatile
OpenAI embeddings	proxies conflict with new openai	Switched to HuggingFace local embeddings

Environment Variables

The .env file must be in the project root. Required variables:

```
GROQ_API_KEY=gsk_your_key_here
OPENAI_API_KEY=dummy
OPENAI_MODEL=gpt-4o
EMBEDDING_MODEL=text-embedding-3-small
CHROMA_PERSIST_DIR=./vectorstore
```

Get a free Groq API key at console.groq.com/keys. The OpenAI key is not used but must be present to avoid import errors.

First Run — Embedding Model Download

The first time you upload a file, the app downloads the all-MiniLM-L6-v2 model from HuggingFace (~90MB). This is a one-time download. After that, embeddings run fully offline.

6. Daily Usage — How to Start the App

Every time you return to work on this project, run these three commands:

Step	Command	What it does
1	`cd "C:\Users\...\ai-business-insights"`	Navigate to the project folder
2	`venv\Scripts\activate`	Activate the virtual environment
3	`streamlit run app.py`	Start the app at localhost:8501

7. Supported File Types

Format	What gets indexed	Analytics tab
CSV (.csv)	One Document per row + stats summary	Yes — KPIs and charts
Excel (.xlsx, .xls)	Same as CSV	Yes — KPIs and charts
PDF (.pdf)	Text extracted page by page	No
Text (.txt, .md)	Full content as one Document	No